

Aprendizaje no supervisado

Jorge Gallego

Inter-American Development Bank

Bogotá Summer School in Economics — 2026

Agenda de la sesión

- Hasta ahora, aprendizaje **supervisado**: predecir un *outcome* conocido
- Hoy cambiamos de mundo: **no** hay etiquetas, queremos **descubrir estructura**
 1. Introducción al aprendizaje no supervisado
 2. Clustering (k-medias)
 3. Detección de anomalías
 4. Representación de datos y *embeddings*

Parte 1

¿Qué es el aprendizaje no supervisado?

Aprendizaje no supervisado

- En el supervisado teníamos un *outcome* y que queríamos predecir
- En el **no supervisado** no hay *y*: solo tenemos las características
- El objetivo no es predecir, sino **describir** y **descubrir patrones** en los datos
- Ninguna variable es más importante que las demás: las miramos todas en conjunto

Tres tareas que veremos

1. **Clustering**: agrupar objetos similares (segmentar)
2. **Detección de anomalías**: encontrar los casos “raros”
3. **Representación / reducción de dimensión**: resumir muchas variables en pocas, más útiles
 - Como no hay etiquetas, evaluar el resultado es más difícil: mucho depende de la **interpretación** del analista

¿Para qué sirve?

- **Segmentación** de clientes, votantes o mercados
- **Detección de fraude**, fallas o intrusiones (lo raro)
- **Visualización** y **compresión** de datos de alta dimensión
- **Preprocesamiento**: crear mejores variables para un modelo supervisado posterior

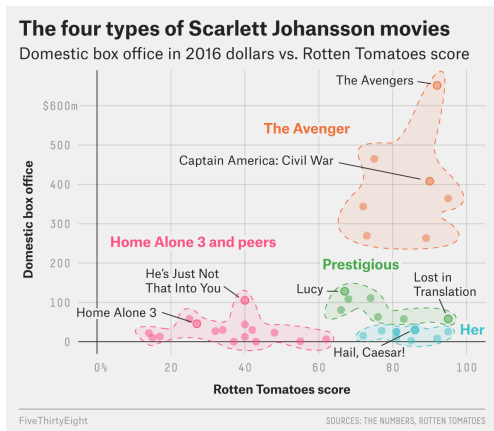
Parte 2

Clustering con k-medias

Clustering

- A veces conviene **agrupar** objetos en categorías que no conocíamos de antemano
- En marketing: segmentar mercados para una estrategia por grupo
- En campañas políticas: identificar tipos de votantes
- El clustering divide los datos en **clusters**: grupos de objetos similares entre sí y distintos de los demás

Un ejemplo cotidiano



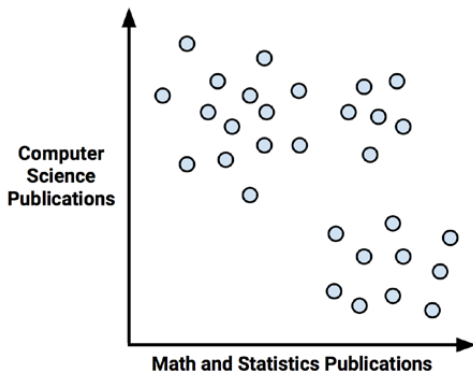
Las películas de un actor podrían agruparse por género sin decirle al algoritmo cuáles son los géneros.

Clustering: clasificación no supervisada

- Es **no supervisado**: no decimos ex ante cómo deben verse los grupos
- No sirve para **predecir**, sino para **descubrir** estructura
- Los grupos que devuelve el computador **no tienen significado explícito**: la interpretación la pone el analista
- ¿Cómo decide el computador dónde están las fronteras de los grupos?

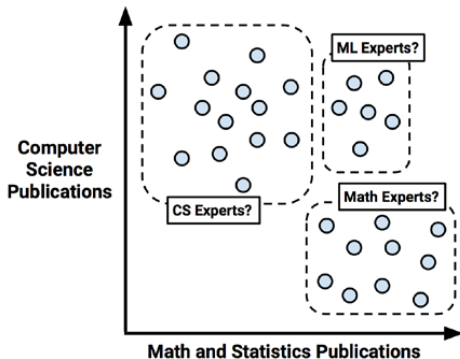
Ejemplo: conferencia de *data science*

- Queremos sentar a los asistentes según su especialidad, pero no sabemos a qué área pertenecen
- Los describimos por dos características: cuánto publican en **computer science** y en **matemáticas/estadística**



Ejemplo: conferencia de *data science*

¿Se ven grupos? Computer scientists, matemáticos/estadísticos, y *machine learning* (mucho de ambos).

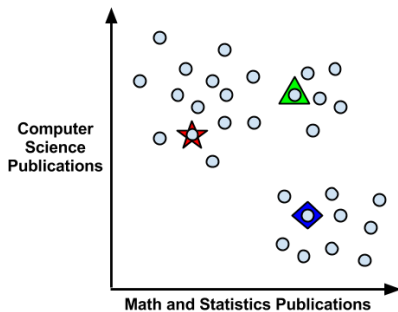


El algoritmo de k-medias

- El más usado para clustering; emparentado con kNN
- Asigna cada uno de los n ejemplos a uno de k clusters
- El número k se fija **ex ante** (de forma heurística)
- Busca **minimizar** las diferencias dentro de cada cluster y **maximizar** las diferencias entre clusters

k-medias: elegir centros

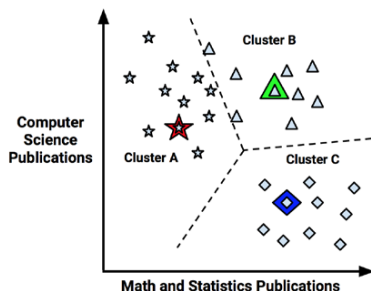
- Se eligen al azar k puntos como centros iniciales (aquí $k = 3$):



k-medias: asignar al centro más cercano

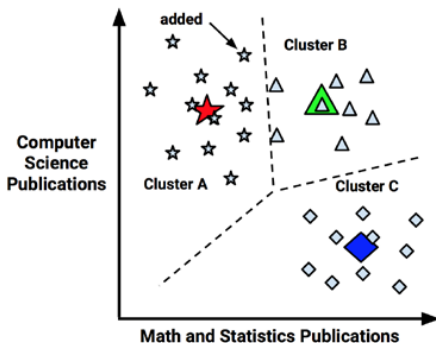
- Cada punto se asigna al centro más cercano (distancia euclídea): se forma el **diagrama de Voronoi**

$$\text{dist}(x, y) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$$



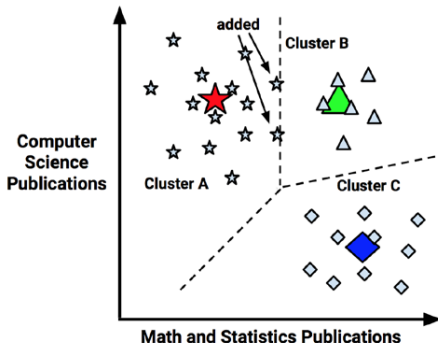
k-medias: actualizar los centroides

- Cada centro se mueve al **centroide** (la posición promedio) de los puntos de su cluster:



k-medias: iterar hasta converger

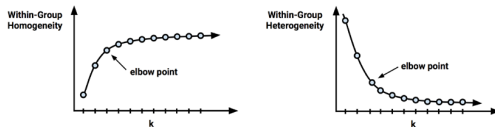
- Con los nuevos centros se reasignan los puntos; algunos cambian de cluster
- Se repite hasta que ya no haya cambios:



¿Cuántos clusters?

- k es una decisión crucial (y el algoritmo también es sensible a los centros iniciales)
- Si hay un **prior**, úsalo (categorías de negocio, número de mesas, de campañas...)
- Regla de dedo: $k = \sqrt{n/2}$
- **Método del codo**: aumentar k hasta que la ganancia marginal en homogeneidad se aplane

Método del codo



Nos quedamos en el “codo”: donde añadir clusters ya casi no reduce la heterogeneidad.

Parte 3

Detección de anomalías

¿Qué es una anomalía?

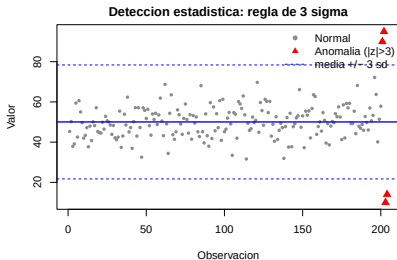
- Una **anomalía** (*outlier*) es un caso que se aparta mucho del patrón general
- A menudo lo raro es lo más interesante:
 - ▶ transacciones **fraudulentas** con tarjeta
 - ▶ **fallas** de una máquina o un sensor
 - ▶ **intrusiones** en una red; errores en los datos
- Suele ser un problema **no supervisado**: no tenemos ejemplos etiquetados de “anomalía”

Cuatro familias de métodos

- **Estadístico:** lo que cae lejos de lo típico (z-score, rango intercuartílico)
- **Por distancia:** puntos lejos de sus vecinos o del centro de su cluster
- **Por densidad:** puntos en zonas poco pobladas (LOF)
- **Por modelo:** aprender la “forma normal” y marcar lo que no encaja (*isolation forest, one-class SVM*)

Enfoque estadístico

- Marcar como atípico lo que cae lejos de lo **típico**
- Univariado: valores con $|z| > 3$ (regla de 3 sigma) o fuera del rango intercuartílico (Tukey)
- Simple y rápido, pero mira **una variable a la vez** y asume una forma (p. ej. normal)



Enfoque por distancia

- Idea: un punto es anómalo si está **lejos** de los demás
- Distancia al k -ésimo vecino más cercano: a mayor distancia, más aislado
- O distancia al **centroide** de su cluster (conecta con k-medias): los puntos lejos de todos los centros son sospechosos
- Limitación: falla cuando hay zonas con densidades muy distintas

LOF: Local Outlier Factor

- Idea: comparar la **densidad local** de un punto con la de sus vecinos
- Si un punto está en una zona **mucho menos densa** que sus vecinos, es sospechoso
- Devuelve un **puntaje**:
 - ▶ $LOF \approx 1$: densidad parecida a la de sus vecinos (normal)
 - ▶ $LOF \gg 1$: mucho más aislado que sus vecinos (anomalía)
- Ventaja sobre el enfoque por distancia: detecta anomalías **locales**, no solo las globales

LOF: ¿cómo se calcula? (intuición)

1. Para cada punto se miran sus k vecinos más cercanos
2. Su **densidad local** se aproxima como el inverso de la distancia promedio a esos vecinos:

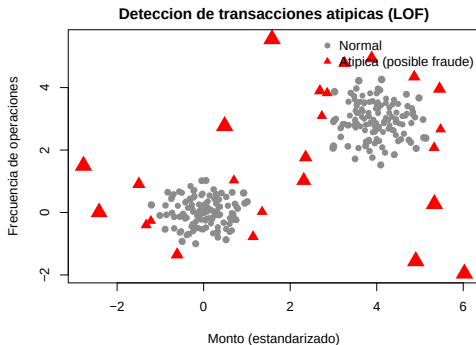
$$\text{densidad}(p) \approx \frac{1}{\text{dist. promedio de } p \text{ a sus } k \text{ vecinos}}$$

3. El **LOF** compara esa densidad con la de sus vecinos:

$$LOF(p) \approx \frac{\text{densidad promedio de los vecinos de } p}{\text{densidad de } p}$$

4. $LOF \approx 1$: normal; $LOF \gg 1$: zona mucho menos densa que la de sus vecinos $\Rightarrow$ anomalía

LOF en acción



Los puntos con LOF alto (rojos) están en zonas poco densas, lejos de los grupos.

Isolation Forest

- Enfoque **por modelo**, basado en árboles (conecta con random forests)
- Construye árboles con **cortes aleatorios**: elige una variable al azar y un punto de corte al azar, y repite
- Cada corte parte los datos en dos; se sigue hasta **aislar** cada punto
- Un punto **anómalo** se aísla con **pocos cortes** (está apartado); uno **normal**, en zona densa, requiere **muchos**
- La anomalía se mide por el número **promedio** de cortes (pocos = anómalo); rápido y escala bien

¿Cómo evaluar sin etiquetas?

- Sin etiquetas de “anomalía”, validar es difícil: a menudo se **inspeccionan a mano** los casos con mayor puntaje
- Si tenemos **algunas** etiquetas (p. ej. fraudes confirmados), es un problema **semi-supervisado**: podemos medir cuántos de los *top-N* marcados eran reales (precisión@N)
- No hay umbral universal: cuántos casos marcar depende del **costo** de cada error
- Como se basa en distancias, conviene **escalar** las variables (como en kNN)

Parte 4

Representación de datos y *embeddings*

El reto de la alta dimensión

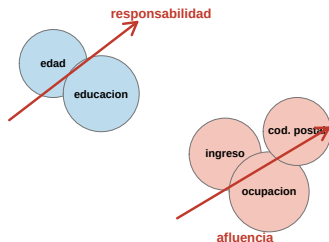
- Muchos problemas tienen **decenas o miles** de variables
- Difíciles de **visualizar**, costosas de procesar, y con la **maldición de la dimensionalidad**
- Pero muchas variables están **correlacionadas**: hay redundancia
- ¿Podemos **resumir** la información en muchas menos dimensiones, perdiendo poco?

Extracción de características

- La **selección** de variables elige un subconjunto; la **extracción** crea variables **nuevas y compuestas**
- Es la versión no supervisada del *feature extraction* de la sesión 4
- La técnica clásica: **Análisis de Componentes Principales** (PCA), que resume muchas variables **correlacionadas** en pocas
- La matemática exacta se sale del curso; vamos por la **intuición**

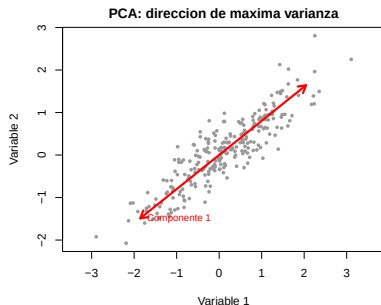
PCA: la intuición

- Para predecir si alguien paga un crédito hay decenas de atributos: edad, educación, ingreso, ocupación, código postal. . .
- Muchos están **correlacionados**: reflejan pocas **dimensiones ocultas** —p. ej. “responsabilidad” y “afluencia”
- PCA **descubre** esas dimensiones compuestas



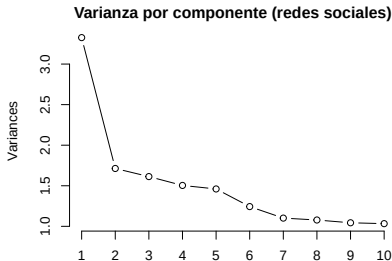
¿Qué hace PCA?

- Resume la **covarianza** de variables correlacionadas en pocos **componentes** (combinaciones lineales de las originales)
- El primero apunta en la dirección de **máxima varianza**; los demás son **ortogonales** y captan lo que queda
- Es **determinista**; los componentes resumen los datos pero **no** tienen un significado directo



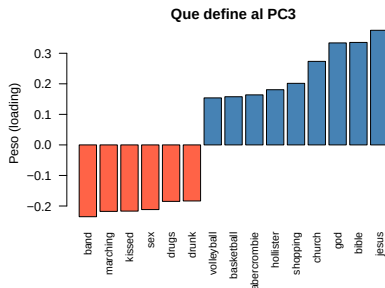
¿Cuántos componentes? (scree plot)

- Cada componente capta una fracción **decreciente** de la varianza
- El **scree plot** muestra ese aporte; nos quedamos en el “codo”
- Ejemplo (redes sociales, 36 palabras): caída fuerte tras el primero; con 5 componentes capturamos ~27% de la varianza



Interpretar con los *loadings*

- Los **loadings** (pesos) dicen cuánto aporta cada variable original a un componente
- Mirándolos le damos **sentido** a un componente, aunque PCA no lo etiquete
- El PC3 de los teens separa **religión** (jesus, bible, church) de **fiesta** (drunk, drugs, sex)



¿Para qué usar PCA?

- **Compresión y visualización** (graficar en 2–3 componentes)
- **Quitar multicolinealidad** antes de una regresión (los componentes no están correlacionados)
- **Preprocesamiento**: con pocas componentes, métodos como kNN o redes neuronales se vuelven factibles
- Costo: los componentes son **menos interpretables** que las variables originales

Embeddings

- Un **embedding** representa un objeto complejo como un **vector denso** de pocos números
- **word2vec**: palabras como vectores; las de significado parecido quedan cerca
- Y las relaciones se vuelven **aritmética**: “ $\text{rey} - \text{hombre} + \text{mujer} \approx \text{reina}$ ”

Embeddings: relaciones con significado



Autoencoders

- Una red neuronal que aprende a **comprimir** los datos y luego **reconstruirlos**
- La capa central (un “cuello de botella”) es una **representación compacta** aprendida
- Es la versión **profunda y no lineal** de la reducción de dimensión
- Las redes neuronales aprenden embeddings de palabras, imágenes, usuarios, productos...
- Es el puente hacia la última sesión: **aprendizaje profundo**

Resumen de la sesión

- El aprendizaje **no supervisado** descubre estructura sin etiquetas
- **Clustering** (k-medias): agrupa objetos similares; elegir k es clave
- **Detección de anomalías**: estadística, distancia, densidad (LOF) o modelos (isolation forest); el contexto define el umbral
- **Representación**: PCA resume muchas variables en pocas componentes; los *embeddings* y *autoencoders* extienden la idea hacia el aprendizaje profundo

Aplicación real en Colombia: UIAF

- La **UIAF** (Unidad de Información y Análisis Financiero) es la unidad de **inteligencia financiera** del país
- Su misión: detectar **lavado de activos** y financiación del terrorismo en un océano de datos (miles de millones de registros)
- No hay una etiqueta de “culpable/inocente”: hay que encontrar lo **atípico** y las **redes** sospechosas

UIAF: resultados y lección

- Prioriza los **reportes de operación sospechosa** (ROS) que se remiten a la Fiscalía
- La efectividad de esos reportes mejoró de forma notable al incorporar analítica de datos
- Ejemplo claro de **aprendizaje no supervisado** al servicio de la lucha antilavado y anticorrupción
- *Fuente:* UIAF (Ministerio de Hacienda); descripciones públicas de sus técnicas de minería de datos y análisis de redes.